GIÁO TRÌNH THỰC HÀNH · LẬP TRÌNH BACKEND

CHƯƠNG 07

UPLOAD HÌNH ẢNH & KẾT NỐI FRONTEND – BACKEND

MULTER · VITE · AXIOS · CORS



01 / ĐỊNH HƯỚNG

Mục tiêu, sản phẩm và điều kiện đầu vào

Bài thực hành phát triển trực tiếp từ Chương 06 (JWT), không tạo project Backend mới và không đổi sang hệ Products minh họa trong đề cương.

Tình huống thực tế

Hệ thống Quản lý sinh viên đã có đăng ký/đăng nhập, phân quyền Admin và 5 API CRUD. Tuy nhiên người dùng chỉ thử qua Postman/Swagger, chưa có giao diện web và chưa thể cập nhật ảnh đại diện. Nhiệm vụ: thêm Avatar an toàn và một Frontend nhỏ gọi API thật.

Sau hai buổi, sinh viên làm được

- Phân biệt application/json và multipart/form-data; biết Multer tạo req.file, FormData tạo nội dung gửi.
- Upload một ảnh JPEG/PNG tối đa 2MB; URL lưu trên dbo.Users, ảnh nằm trong public/uploads.
- Dùng JWT xác thực người tự cập nhật Avatar; giải thích được 401 và 403.
- Dùng Vite + React + Axios dựng giao diện Login, danh sách sinh viên, xóa dữ liệu với Admin, upload Avatar.
- Cấu hình CORS đúng Origin; dùng Network, Postman và SSMS để chứng minh dữ liệu thật.

Đầu vào phải có

Thành phần	Yêu cầu kiểm chứng
Project	backend-quan-ly-sinh-vien; có pnpm dev/build, 5 CRUD SinhVien và Swagger YAML.
Auth	POST /api/auth/register, /login; GET /api/auth/profile; isAuth và isAdmin đã chạy.
Database	SQL Server + Sequelize có dbo.Users tách biệt dbo.SinhVien.
Công cụ	VS Code, Node.js, pnpm, SSMS, Postman, trình duyệt có DevTools.

ĐIỂM DỪNG CẦN XÁC MINH

Bắt đầu ở trạng thái pnpm build thành công. Nếu project Chương 06 còn lỗi phải ghi lại và sửa trước khi sang phần upload.

02 / HỢP ĐỒNG API

Giữ nguyên API cũ, chỉ mở rộng tối thiểu

Tài liệu Chương 06 quy định GET SinhVien public, thao tác ghi chỉ Admin và cấu trúc JSON success/message/data. Các yêu cầu này tiếp tục áp dụng.

Method	Endpoint	Quyền / kết quả
POST	/api/auth/register	Public; 201/400/409; không cho chọn role.
POST	/api/auth/login	Public; 200 + accessToken; sai 401.
GET	/api/auth/profile	isAuth; 200 hoặc 401; hồ sơ chính mình.
GET	/api/v1/sinh-vien	Public; 200; data là mảng.
GET	/api/v1/sinh-vien/:maSV	Public; 200 hoặc 404.
POST	/api/v1/sinh-vien	isAuth + isAdmin; 201/400/409.
PUT	/api/v1/sinh-vien/:maSV	isAuth + isAdmin; 200/400/404.
DELETE	/api/v1/sinh-vien/:maSV	isAuth + isAdmin; 200/404.
POST (mới)	/api/auth/me/avatar	isAuth; thành công 200 và trả avatarUrl.

Hợp đồng mới thống nhất cho bài lab

- Form field file mang tên avatar; chỉ nhận 1 file, JPEG/PNG, tối đa 2 × 1024 × 1024 byte.
- Tên cột thêm: dbo.Users.avatarUrl NVARCHAR(255) NULL (chỉ khi chưa có); không đưa Avatar vào dbo.SinhVien.
- Thành công: {success:true, message:"Cập nhật ảnh đại diện thành công", data:{avatarUrl:"/uploads/..."}}.
- Không file/sai loại/sai nội dung: 400; quá 2MB: 413; thiếu/sai JWT: 401; lỗi server: 500.
- Đường dẫn ví dụ PORT 5000; nếu project dùng PORT khác, cập nhật đúng nơi cấu hình.

KHÔNG ĐƯỢC LÀM

Không sửa hợp đồng response của API cũ, không bỏ isAuth/isAdmin để Frontend gọi được, không lưu file nhị phân trực tiếp vào SQL Server trong bài này.

03 / KIẾN TRÚC

Hai luồng Request xuyên suốt Chương 07

Giải thích được Request đang ở đâu, Middleware nào làm gì và thông tin nào được lưu trong SQL Server.

Luồng A — Cập nhật Avatar của tài khoản đang đăng nhập

```
React <input type="file">
  → FormData.append("avatar", file)
  → Axios POST + Authorization: Bearer <JWT>
  → Express Route → isAuth → Multer → Controller
  → Service → cập nhật dbo.Users.avatarUrl qua Sequelize
  → File vật lý public/uploads/  +  SQL chỉ giữ /uploads/...
  → React nhận URL và hiển thị ảnh
```

Luồng B — Xem và xóa SinhVien trên giao diện

```
React → Axios GET /api/v1/sinh-vien
  → Express → Service → Sequelize → SQL Server
  → JSON { success, message, data: [...] }
  → React render Table / Loading / Error

Nút Xóa (Admin) → Axios DELETE + Bearer Token
  → isAuth → isAdmin → Controller → Service → Database
```

Cần phân biệt

Khái niệm	Điểm ghi nhớ
express.json()	Đọc JSON; không tự ghi file gửi multipart/form-data.
Multer	Middleware nhận file multipart; req.file là metadata file.
FormData	Đóng gói dữ liệu file ở Frontend; field name phải khớp upload.single("avatar").
CORS	Trình duyệt kiểm tra quyền cross-origin, không phải cơ chế xác thực.
JWT / RBAC	isAuth xác minh danh tính; isAdmin xét quyền thao tác ghi.

ĐIỂM DỪNG CẦN XÁC MINH

Sinh viên phải vẽ lại bằng tay/giải thích được một luồng upload, một luồng GET trước khi yêu cầu AI viết mã.

04 / BẢN ĐỒ FILE

Phân biệt file tạo mới và file cần sửa

Tên file bên dưới là gợi ý; nếu project đang có tên khác, giữ nguyên cấu trúc thật và chỉ ra file tương ứng.

```
backend-quan-ly-sinh-vien/
  public/uploads/                    # Ảnh hợp lệ được phục vụ tĩnh
  sql/05-add-avatar-url.sql          # Script SQL không phá dữ liệu
  src/
    app.ts hoặc server.ts            # cors(), express.static()
    models/user.model.ts             # Thêm ánh xạ avatarUrl (nếu cần)
    middlewares/upload.middleware.ts # Cấu hình Multer + giới hạn
    routes/auth.routes.ts            # Gắn POST /me/avatar + isAuth
    controllers/auth.controller.ts   # Handler Avatar (hoặc file riêng)
    services/auth.service.ts         # Cập nhật đúng User từ JWT
    middlewares/error.middleware.ts  # Đổi MulterError thành JSON
  docs/openapi.yaml                 # Bổ sung API Avatar, Bearer
  docs/prompt-log.md / test-results.md
  frontend/                         # App Vite/React độc lập
    src/api/client.ts                # Axios baseURL
    src/components/                 # Login, SinhVienTable, AvatarUpload
    src/App.tsx
    .env.example                     # VITE_API_BASE_URL=...
```

Ba quy tắc không thay đổi

- Backend: giữ ESM/NodeNext, pnpm, import nội bộ đuôi .js nếu cấu hình cũ đang dùng.
- SQL Server: chỉ thay đổi schema bằng script đã review; tuyệt đối không dùng sync({force:true})/sync({alter:true}).
- Frontend nằm trong thư mục frontend/ của cùng repository; có package.json riêng nhưng không tạo một Backend thứ hai.

BẰNG CHỨNG CẤU TRÚC

Chụp cây thư mục trước/sau, lưu trong báo cáo. Không nộp node_modules, dist, public/uploads chứa dữ liệu riêng, hoặc .env có bí mật thật.

05 / BƯỚC 0

Khảo sát dự án và chốt trạng thái ban đầu

Không yêu cầu AI sửa hàng loạt ngay khi mở project. Đầu tiên phải xác định các ràng buộc từ Chương 06 và sao lưu.

Thao tác tại thư mục Backend (có package.json)

```
node -v
pnpm -v
pnpm install
pnpm build
pnpm dev
```

01 Mở /api-docs và kiểm tra GET /api/v1/sinh-vien trả 200 với data là mảng.

02 Dùng Student login và gọi GET /api/auth/profile có JWT; Admin gọi DELETE trên bản ghi test có quyền.

03 Mở SSMS đối chiếu bảng dbo.Users và các cột đang tồn tại; ghi nhận PK/id, Role, timestamps.

04 Commit/sao lưu phiên bản chạy được; không dùng tài khoản thật hoặc ảnh riêng tư để test.

05 Gửi AI file package.json, tsconfig, app/server, auth route, User Model, auth service, error handler và YAML (đã bỏ bí mật).

PROMPT 0 · Chỉ khảo sát, chưa tạo code

C · CONTEXT Tôi tiếp tục đúng backend-quan-ly-sinh-vien sau Chương 06: Express + TypeScript + pnpm + Sequelize/SQL Server + JWT + Zod + Swagger YAML.

T · TASK Đọc những file tôi cung cấp và xác định vị trí phù hợp để thêm upload Avatar và CORS; kiểm tra Schema bảng Users.

C · CONSTRAINT Chưa viết code; không tạo Backend khác; không đoán file/schema chưa có; không yêu cầu .env hoặc token thật.

O · OUTPUT Bảng file/đang làm gì/cần sửa/lý do, cùng các thông tin còn thiếu.

ĐIỂM DỪNG CẦN XÁC MINH

Có bản sao project và nhật ký các API đã test. Nếu pnpm build không thành công thì chưa chuyển Bước 1.

06 / BƯỚC 1

Bổ sung avatarUrl an toàn trong SQL Server

Hồ sơ đăng nhập nằm ở bảng dbo.Users của Chương 06; không chuyển sang bảng dbo.SinhVien và không tạo bảng Users lần hai.

Script SQL tham chiếu (đọc kỹ schema thực tế trước khi chạy)

```
IF OBJECT_ID(N'dbo.Users', N'U') IS NULL
    THROW 50001, N'Chưa có bảng dbo.Users', 1;
GO
IF COL_LENGTH('dbo.Users', 'avatarUrl') IS NULL
BEGIN
    ALTER TABLE dbo.Users
    ADD avatarUrl NVARCHAR(255) NULL;
END;
GO
SELECT TOP (10) id, email, avatarUrl FROM dbo.Users;
```

Cần thực hiện

- Lưu SQL ở sql/05-add-avatar-url.sql; đối chiếu đúng tên bảng/khóa chính và cách viết hoa chữ.
- Nếu avatarUrl đã tồn tại: kiểm tra kiểu dữ liệu, không tự ADD thêm cột mới.
- Bổ sung User Model ánh xạ avatarUrl nullable; giữ tên bảng, timestamps, kiểu ID và các cột auth cũ.
- Chạy script thử lần hai để chứng minh không tạo cột trùng; kiểm tra các User cũ không bị xóa.

PROMPT 1 · SQL + User Model

C · CONTEXT Dự án đã có dbo.Users từ bài JWT gồm id, fullName, email, passwordHash, role.

T · TASK Tạo bản vá SQL và ánh xạ avatarUrl kiểu nullable vào User Model của project.

C · CONSTRAINT Không DROP bảng, không sync force/alter, không sửa SinhVien, không ảnh hưởng register/login và trường hash.

O · OUTPUT Script chạy SSMS, diff User Model, lệnh SELECT xác minh; nêu mọi giả định schema.

ĐIỂM DỪNG CẦN XÁC MINH

SSMS cho thấy avatarUrl là nullable; số User giữ nguyên; pnpm build vẫn chạy được.

07 / BƯỚC 2

Cấu hình Multer: chỉ nhận ảnh hợp lệ ≤ 2MB

Dùng Multer ở Express để xử lý multipart/form-data. Tên field avatar bắt buộc khớp với FormData phía giao diện.

Các quy tắc kỹ thuật bắt buộc

- Cài multer và type của Multer tương thích TypeScript; không đổi pnpm sang npm.
- upload.single("avatar") nhận đúng một file; limits.fileSize = 2 * 1024 * 1024 (2.097.152 byte).
- Chính sách chỉ cho phép .jpg/.jpeg/.png và dạng JPEG/PNG; không chỉ tin phần mở rộng hoặc mimetype client khai báo.
- Kiểm tra chữ ký/nội dung thực của file trước khi chấp nhận làm ảnh hợp lệ; file sai phải được xóa, không để lại bản rác.
- Tên file do server sinh duy nhất và an toàn (VD randomUUID), tránh ghi đè hoặc dùng nguyên đường dẫn do client gửi.
- Nếu lưu tạm để kiểm tra trước, thư mục tạm không được public; sau khi hợp lệ mới chuyển vào public/uploads.
- Khi Multer báo LIMIT_FILE_SIZE, trả 413 theo quy ước lab, không trả 500 chung chung.

Đoạn tham chiếu cần đọc hiểu

```
const MAX_BYTES = 2 * 1024 * 1024;
const upload = multer({
  storage,
  limits: { fileSize: MAX_BYTES }
});
// Kiểm tra nội dung file thực + dọn file lỗi ở bước sau.
```

PROMPT 2 · Multer và kiểm tra an toàn

C · CONTEXT Tôi cần upload Avatar cho User trên Express TypeScript hiện có.

T · TASK Viết Multer middleware lưu ảnh hợp lệ vào public/uploads, giới hạn 2MB, JPEG/PNG và kiểm tra nội dung thực.

C · CONSTRAINT isAuth chạy trước Multer; filename an toàn; dọn file lỗi; không public thư mục tạm; ESM/NodeNext đúng project.

O · OUTPUT Danh sách file cài/sửa, code đúng file, giải thích req.file, bảng test kích thước/định dạng.

ĐIỂM DỪNG CẦN XÁC MINH

Chưa cần cập nhật DB. Chứng minh upload xử lý được file hợp lệ, từ chối file sai và không giữ file rác.

08 / BƯỚC 3

Tạo POST /api/auth/me/avatar và cập nhật đúng User

Route minh họa trong slide là /me/avatar. Trong bài này mount trong auth router để có URL đầy đủ /api/auth/me/avatar.

Thứ tự Middleware và Controller

```
router.post(
  "/me/avatar",
  isAuth,
  upload.single("avatar"),
  uploadAvatar
);
```

- Controller kiểm tra req.file; thiếu file trả JSON lỗi 400 theo hợp đồng đã chốt.
- Lấy user ID từ req.user sau khi isAuth xác minh JWT, tuyệt đối không dùng req.body.userId.
- Service cập nhật avatarUrl = "/uploads/<ten-file-hop-le>" bằng Sequelize, chỉ tại User đăng nhập.
- Response 200 chứa đường dẫn URL tương đối trong data.avatarUrl và giữ success/message/data.
- Khi cập nhật DB thất bại: xử lý lỗi và dọn file mới vừa ghi để giảm file mồ côi.
- Không trả passwordHash hoặc thuộc tính nội bộ trong API Profile hay response upload.

Dùng Express phục vụ ảnh tĩnh

```
app.use(
  "/uploads",
  express.static("public/uploads")
);
// Chỉ public thư mục ảnh hợp lệ đã kiểm tra.
```

PROMPT 3 · Route, Controller, Service Avatar

C · CONTEXT Chương 06 đã có /api/auth/profile, User Model, isAuth và response success/message/data.

T · TASK Thêm POST /api/auth/me/avatar và cập nhật User.avatarUrl theo req.user, trả 200 + URL.

C · CONSTRAINT Giữ route auth cũ, không tin userId từ client; không lộ hash; xử lý DB lỗi và không để ảnh rác.

O · OUTPUT Diff các file, code đầy đủ đúng từng file, sơ đồ request và 6 cách test.

ĐIỂM DỪNG CẦN XÁC MINH

Upload Student A không thể thay Avatar Student B; GET profile của A thấy URL mới, profile B không đổi.

09 / BƯỚC 4

Thử Upload bằng Postman trước khi tạo Frontend

Chỉ qua được bước này mới bắt đầu React. Phân biệt Body form-data có file với Body raw JSON.

Request chính xác trong Postman

Mục	Cấu hình / giá trị
Method / URL	POST {{baseUrl}}/api/auth/me/avatar
Authorization	Bearer {{studentToken}} (Token lấy từ Login)
Body	form-data; key avatar; chọn loại File; chọn ảnh .png/.jpg/.jpeg
Không gửi	Không gửi userId để chỉ định người khác; không gửi body JSON
Kỳ vọng	200, success=true, data.avatarUrl là /uploads/<file>

Ba lớp bằng chứng cho một lần upload

01 Postman: status 200, trường data.avatarUrl đúng quy ước.

02 Ổ đĩa: xuất hiện file ảnh mới trong public/uploads, tên không trùng file trước đó.

03 SQL Server: SELECT avatarUrl của đúng User; xem ảnh bằng http://localhost:5000/uploads/<filename>.

Kiểm tra lỗi trước khi chuyển bước

- Không JWT: 401 và không có file mới; body field photo thay avatar: bị từ chối.
- Ảnh dung lượng lớn hơn 2MB: 413; file TXT đổi đuôi thành .png vẫn phải bị từ chối.
- Chụp kết quả trong docs/test-results.md: Expected, Actual, ảnh chứng minh và ghi nhận số lượng file trước/sau.

PROMPT 4 · Lập bộ kiểm thử Upload

C · CONTEXT Tôi đã có POST /api/auth/me/avatar với JWT + Multer + SQL Server.

T · TASK Tạo hướng dẫn kiểm thử trên Postman cho thành công, thiếu JWT, sai field, >2MB, giả mạo đuôi, DB lỗi.

C · CONSTRAINT Không tự kết luận PASS; không sinh token giả; chỉ dùng ảnh và User test.

O · OUTPUT Danh sách request, expected status, bước kiểm tra file/SSMS và cách xác nhận không có file rác.

10 / BƯỚC 5

Swagger YAML và công khai ảnh tĩnh đúng mức

Cập nhật chính docs/openapi.yaml đang dùng ở Chương 06, không chuyển sang cơ chế Swagger JSDoc mới.

Khai báo cho API Upload mới

- Method/path: POST /api/auth/me/avatar, tags Auth hoặc Upload theo cấu trúc YAML cũ.
- security: BearerAuth; requestBody: multipart/form-data với property avatar kiểu string, format: binary.
- Ví dụ 200 chứa data.avatarUrl; mô tả 400, 401, 413 và lỗi 500 theo hợp đồng.
- Đảm bảo ảnh tĩnh /uploads/<tên file> mở được từ trình duyệt sau upload, không public toàn bộ thư mục project.
- GET /api/auth/profile trả thêm avatarUrl nếu Model/Service đang trả hồ sơ; tuyệt đối không thêm passwordHash.

Mô tả OpenAPI tối thiểu (không chép đè toàn YAML)

```yaml
requestBody:
  required: true
  content:
    multipart/form-data:
      schema:
        type: object
        required: [avatar]
        properties:
          avatar:
            type: string
            format: binary
```

PROMPT 5 · Bổ sung Swagger YAML

C · CONTEXT Project đã dùng docs/openapi.yaml và swagger UI /api-docs từ các chương trước.

T · TASK Bổ sung phần mô tả POST /api/auth/me/avatar nhận multipart, BearerAuth và các response.

C · CONSTRAINT Giữ YAML và component securitySchemes hiện tại; không tạo lại file, không xóa các API CRUD.

O · OUTPUT Chỉ trả phần YAML cần ghép, vị trí tương ứng và checklist kiểm thử Swagger.

ĐIỂM DỪNG CẦN XÁC MINH

Swagger Try it out upload thành công bằng JWT; GET/POST/PUT/DELETE cũ vẫn xuất hiện và chạy theo phân quyền.

11 / BƯỚC 6

Khởi tạo Frontend Vite + React và thiết lập CORS

Frontend có project riêng trong frontend/, nhưng vẫn giao tiếp với cùng Backend Node.js và SQL Server cũ.

Các thao tác chuẩn bị

```
pnpm create vite frontend --template react-ts
cd frontend
pnpm install
pnpm add axios
pnpm dev
```

Nếu thư mục frontend đã tồn tại, không chạy lại lệnh create; đối chiếu cấu trúc và giữ phần đang có. Vite thường chạy localhost:5173; phải xem Terminal để xác định cổng thực.

Phía Backend: chỉ cho phép Origin của Vite đang chạy

```
app.use(cors({
  origin: "http://localhost:5173"
}));
```

Phía Frontend: biến môi trường công khai

```
# frontend/.env.example
VITE_API_BASE_URL=http://localhost:5000
```

- Chỉ để URL API công khai trong biến VITE_; không đưa JWT_SECRET, DB_PASSWORD hoặc credential vào Frontend.
- Nếu browser lỗi CORS: kiểm tra Origin, thứ tự middleware và response preflight; không gỡ isAuth, không sửa JWT để chữa CORS.
- Postman gọi được không chứng minh trình duyệt qua CORS; phải mở DevTools → Network để xác nhận.

PROMPT 6 · Vite React + Axios + CORS

C · CONTEXT Backend Express TypeScript chạy localhost:5000; Frontend React TypeScript dự kiến localhost:5173.

T · TASK Tạo Vite trong frontend và client Axios dùng VITE_API_BASE_URL; thêm cors theo allowlist phía Backend.

C · CONSTRAINT Không thêm server/DB khác; không dùng wildcard CORS để sửa lỗi; không lộ secret trong Vite.

O · OUTPUT Lệnh chạy, file cần tạo/sửa, code gọn và cách test từ trình duyệt.

12 / BƯỚC 7

Hiển thị danh sách SinhVien bằng Axios GET

Dùng đúng endpoint quản lý SinhVien, không dùng /api/products và không dùng dữ liệu mock để giả vờ tích hợp.

API thật đã có từ Chương 04

```
GET http://localhost:5000/api/v1/sinh-vien

{
  "success": true,
  "message": "...",
  "data": [{ "maSV": "SV001", "hoTen": "...",
             "email": "...", "maLop": "..." }]
}
```

Yêu cầu Component SinhVienTable

- Gọi API bằng Axios khi trang mở; lấy mảng tại response.data.data, không tự giả định response.data chính là mảng.
- Hiển thị Table 4 cột maSV, hoTen, email, maLop; hiển thị lời nhắc rõ khi danh sách rỗng.
- Có Loading lúc chờ; Error khi Backend tắt, bị lỗi hoặc response không đúng; có nút Thử lại.
- Không viết connection string SQL Server ở Frontend; dữ liệu phải đến từ GET qua HTTP.
- DevTools Network thấy request đúng URL, status 200; khi thêm User mới qua Backend, Reload danh sách có dữ liệu mới.

PROMPT 7 · React Table + Loading + Error

C · CONTEXT Tôi có Vite React TypeScript + Axios. Backend thật trả {success,message,data} tại /api/v1/sinh-vien.

T · TASK Viết SinhVienTable gọi GET bằng Axios; render 4 cột, Loading, Error, Empty và nút Thử lại.

C · CONSTRAINT Không dùng mock products; không gọi SQL trực tiếp; giữ TypeScript, tránh lỗi setState sau unmount.

O · OUTPUT Component + type dữ liệu + nơi import vào App + cách test Network khi server bật/tắt.

ĐIỂM DỪNG CẦN XÁC MINH

Dữ liệu UI thay đổi tương ứng với dữ liệu thật của SQL Server. Khi tắt Backend, UI hiện Error thay vì trang trắng.

13 / BƯỚC 8

Đăng nhập qua React và sử dụng Bearer JWT

Tái sử dụng POST /api/auth/login từ Chương 06; không tạo API login mới. Giữ cơ chế 401/403 ở Backend.

Yêu cầu màn hình Login

- Có input email/password; nút Đăng nhập và thông báo trạng thái gửi request.
- POST /api/auth/login dạng JSON; đọc token đúng vị trí response đã xác nhận từ Chương 06.
- Dùng token trong bộ nhớ ứng dụng cho bài lab; reload sẽ cần login lại. Nếu chọn nơi lưu khác, giải thích rủi ro XSS.
- Gọi GET /api/auth/profile với Authorization: Bearer <token> và render fullName, role, avatarUrl.
- Logout xóa trạng thái token; khi nhận 401 phải hướng dẫn đăng nhập lại, không tự bỏ qua lỗi.

Header chuẩn — minh họa

```
const res = await axios.get(
  `${API_BASE_URL}/api/auth/profile`,
  { headers: { Authorization: `Bearer ${token}` } }
);
```

Phân biệt đúng

Trạng thái	Frontend phải hiển thị
200	Thông tin tài khoản đã xác thực.
401	Token thiếu/sai/hết hạn: yêu cầu đăng nhập lại.
403	Đã đăng nhập nhưng không đủ quyền thao tác.
CORS / Network	Lỗi kết nối hoặc quyền Origin; không đồng nhất với 401.

PROMPT 8 · Login và Profile qua API cũ

C · CONTEXT Backend có POST /api/auth/login trả JWT 1 giờ và GET /api/auth/profile yêu cầu isAuth.

T · TASK Viết form Login React + trạng thái token trong memory; Axios gọi Profile có Bearer header.

C · CONSTRAINT Không hardcode token hoặc user; không đưa secret vào frontend; đọc đúng JSON contract project.

O · OUTPUT Component, state flow, thông báo 401 và các ca test login đúng/sai/logout.

14 / BƯỚC 9

Nút Xóa sinh viên: chỉ Admin thực thi

Nút Xóa là thao tác DELETE của bài cũ Chương 04, kết hợp phân quyền Admin đã xây dựng ở Chương 06.

Giao diện cần có

- Bảng SinhVien hiển thị thêm nút Xóa ở mỗi dòng khi profile.role = admin; Student không thấy nút này.
- Trước khi gọi DELETE, hiển thị xác nhận rõ mã sinh viên cần xóa; chỉ xóa bản ghi lab.
- Gửi DELETE /api/v1/sinh-vien/:maSV với Bearer JWT đang đăng nhập.
- 200: thông báo thành công và gọi lại GET để danh sách cập nhật; 404: bản ghi không tồn tại.
- 401: yêu cầu login lại; 403: thông báo không đủ quyền; không xóa dòng khỏi UI khi server trả lỗi.
- Bảo vệ thật nằm ở isAuth + isAdmin Backend, không phải chỉ điều kiện hiển thị nút.

Trình tự kiểm thử bắt buộc

Lần thử	Vai trò	Expected
1	Chưa login	Không bấm xóa thành công; API DELETE trực tiếp trả 401.
2	Student	Không có nút; ép gọi DELETE qua Network/Postman trả 403.
3	Admin	Bấm xóa SV test → 200 → GET mới không còn bản ghi.
4	Admin	Xóa lại cùng maSV → 404; UI giữ dữ liệu nhất quán.

PROMPT 9 · Nút Delete có JWT và RBAC

C · CONTEXT SinhVienTable đã có GET; Backend DELETE cũ bắt buộc isAuth + isAdmin.

T · TASK Bổ sung nút Xóa cho Admin, hỏi xác nhận, gọi Axios DELETE có Bearer, refresh dữ liệu sau 200.

C · CONSTRAINT Không ẩn lỗi 401/403/404; không coi ẩn nút là bảo mật; chỉ xóa bản ghi thử.

O · OUTPUT Diff component, bảng status xử lý và test 4 vai trò/tình huống.

15 / BƯỚC 10

Upload Avatar trực tiếp trên giao diện React

Ghép các công cụ thành đúng luồng ở slide Chương 07: chọn ảnh → FormData → Axios + JWT → Multer → SQL Server → UI.

Yêu cầu AvatarUpload component

- Dùng input type=file accept="image/png,image/jpeg" để hỗ trợ chọn ảnh (Backend vẫn phải kiểm tra).
- Kiểm tra sơ bộ dung lượng 2MB ở Frontend để báo lỗi sớm, nhưng không bỏ kiểm tra Backend.
- Tạo FormData; append("avatar", file); gửi POST /api/auth/me/avatar với Bearer JWT.
- Không tự đặt Content-Type multipart/form-data thiếu boundary khi gửi FormData từ trình duyệt.
- Có trạng thái đang tải, thành công/thất bại; sau 200 gọi lại Profile để hiển thị avatarUrl từ SQL Server.
- Hiển thị ảnh bằng API_BASE_URL + avatarUrl; kiểm tra URL là path ảnh được Backend cấp, không nối dữ liệu tùy ý.

Đoạn code tối thiểu cần giải thích

```
const formData = new FormData();
formData.append("avatar", file);

await axios.post(
  `${API_BASE_URL}/api/auth/me/avatar`,
  formData,
  { headers: { Authorization: `Bearer ${token}` } }
);
```

PROMPT 10 · FormData + Axios Avatar

C · CONTEXT Frontend đã login được và có token; Backend upload /api/auth/me/avatar đã kiểm thử trên Postman.

T · TASK Tạo AvatarUpload với chọn ảnh, kiểm tra 2MB, FormData, gửi Axios JWT, loading/error và refresh Profile.

C · CONSTRAINT Field là avatar; browser tự thêm multipart boundary; không gửi userId; không thay API Backend cũ.

O · OUTPUT Component, vị trí gắn vào App, bằng chứng Network + file trên đĩa + SELECT SQL Server.

ĐIỂM DỪNG CẦN XÁC MINH

UI hiển thị ảnh vừa tải lên; profile sau refresh trả URL mới và dữ liệu tồn tại qua restart Backend.

16 / TEST UPLOAD

Ma trận 12 ca kiểm thử Multer, JWT và lưu trữ

Chạy các ca U01–U12 bằng Postman và đối chiếu ở ổ đĩa/SSMS. Không đánh PASS nếu chỉ thấy một response JSON đẹp.

Mã	Tình huống	Expected	Bằng chứng thêm
U01	Avatar PNG hợp lệ + Student JWT	200	File, URL, SQL cập nhật đúng User.
U02	Avatar JPEG hợp lệ + Student JWT	200	Ảnh trình duyệt mở được.
U03	Không có Authorization	401	Không sinh file và không UPDATE.
U04	JWT sai/hết hạn	401	Không có URL mới.
U05	Body thiếu file	400	Không cập nhật SQL.
U06	Field photo thay vì avatar	400	Bị từ chối; không ghi file hợp lệ.
U07	Kích thước > 2MB	413	Bắt LIMIT_FILE_SIZE; không file rác.
U08	TXT đổi tên thành avatar.png	400	Kiểm chữ ký file thực.
U09	Đuôi .gif / MIME không chấp nhận	400	Không tồn tại file công khai mới.
U10	Upload hai lần cùng tài khoản	200	URL mới; không ghi đè bất ngờ.
U11	Student A gửi userId của B trong body	200*	Chỉ A đổi avatar; không tin userId.
U12	Restart server rồi GET Profile	200	URL giữ nguyên, file hiển thị.

* U11: nếu backend từ chối trường userId lạ bằng 400 thì cũng đạt mục tiêu bảo mật; điều bắt buộc là avatar của B không thay đổi. Kiểm tra bằng SELECT và Profile cả hai tài khoản.

CÁCH GHI TEST-RESULTS.MD

Mã ca | Method/URL | Dữ liệu kiểm thử | Expected | Actual | PASS/FAIL | Link/chụp bằng chứng | Lỗi và cách sửa

17 / TEST FRONTEND

Ma trận 12 ca kiểm thử Vite, Axios, CORS và phân quyền

Dùng trình duyệt + Network để kiểm tra F01–F12. Chạy thử ít nhất trên Chrome hoặc Edge ở Origin thật của Vite.

Mã	Tình huống	Expected	Cách xác minh
F01	Vite chạy, Backend bật	Trang mở	Network GET thành công.
F02	GET danh sách có dữ liệu	Table	Bốn cột đúng nội dung SQL.
F03	GET data = []	Empty	Không crash, có thông báo.
F04	Backend tắt khi GET	Error	Không vòng loading vô hạn.
F05	Bật lại Backend, bấm thử lại	200	Bảng hiển thị trở lại.
F06	Login đúng student	200	Profile Student thực từ JWT.
F07	Login sai mật khẩu	401	Không lưu token giả.
F08	GET Profile thiếu token	401	Yêu cầu login, không lộ hồ sơ.
F09	Student ép gọi DELETE	403	DB không đổi; không tin UI.
F10	Admin bấm DELETE bản ghi test	200	Reload GET loại đúng maSV.
F11	Avatar hợp lệ upload từ React	200	Network multipart; ảnh hiện; DB URL.
F12	Origin khác Origin allowlist	CORS chặn	Browser blocked; Postman khác browser.

KIỂM THỬ HỒI QUY

Chạy lại các ca cốt lõi Chương 06: A08/A10 (Profile 401/200), B01 (GET public), B04 (Student DELETE 403), B09 (Admin DELETE 200) và Zod 400 ở POST/PUT. Không đổi expected để che lỗi.

18 / SECURITY REVIEW

Phản biện đoạn code AI nguy hiểm

Đoạn minh họa bên dưới cố tình sai. Không chạy và không ghép vào project; sinh viên phải nêu rủi ro, đề xuất cách sửa và test chứng minh.

```
// CODE SAI — CHỈ ĐỌC ĐỂ REVIEW
app.use(cors());
router.post("/me/avatar", upload.single("avatar"),
  async (req, res) => {
    const fileName = req.file?.originalname;
    await User.update({ avatarUrl: fileName },
      { where: { id: req.body.userId } });
    res.json({ success: true });
  }
);

// Frontend: luôn gửi userId theo ô input; không gửi JWT.
```

Lỗi	Hệ quả / điều cần sửa
Bỏ isAuth	Bất kỳ client nào có thể upload; phải kiểm tra JWT trước Multer.
Tin req.body.userId	Có thể sửa tài khoản khác; lấy id từ JWT đã xác minh.
Dùng nguyên originalname	Có nguy cơ trùng tên và tên không an toàn; dùng tên server tạo.
Không giới hạn size	Có thể tiêu tốn dung lượng; giới hạn 2MB.
Không kiểm nội dung	File giả mạo PNG có thể lọt; kiểm MIME/signature thực.
CORS không giới hạn	Origin không được kiểm soát theo chính sách lab; CORS không phải auth.
Lưu chỉ originalname	Không đúng /uploads/<file> và không xác minh tệp thật.

PROMPT 11 · Review bảo mật code AI

C · CONTEXT Tôi có đoạn upload/CORS/React sinh bởi AI cần security review.

T · TASK Chỉ ra lỗ hổng về JWT, quyền, upload file, path, kiểm tra nội dung, giới hạn size, CORS và response.

C · CONSTRAINT Không bỏ auth để sửa lỗi; chỉ đề xuất thay đổi nhỏ và test được; không dùng mật khẩu/token thật.

O · OUTPUT Bảng file/dòng - rủi ro - bản vá - ca test phải thêm; viết phần giải thích 8–10 câu.

19 / LỖI THƯỜNG GẶP

Sửa đúng nguyên nhân, không viết lại toàn project

Ghi lại method + URL + Origin + status + response + lỗi Terminal/Network; nhờ AI sửa tối thiểu rồi chạy lại ca từng thất bại.

Triệu chứng	Ưu tiên kiểm tra
400 Unexpected field	formData.append("avatar") có khớp upload.single("avatar")?
req.file undefined	Body Postman là form-data/File? Middleware Multer đúng Route và field chưa?
413 File Too Large	fileSize 2MB, logic bắt LIMIT_FILE_SIZE, ảnh có lớn hơn 2.097.152 byte?
500 sau Upload	Thư mục có quyền ghi? User Model và cột avatarUrl đã khớp SQL?
Ảnh 404 sau khi upload	express.static mount /uploads đã đúng? URL /uploads/... trỏ tới file vật lý?
UI báo CORS	Browser Origin chính xác 5173? Middleware cors có trước route? OPTIONS bị chặn?
Network ERR_CONNECTION_REFUSED	Backend có chạy đúng PORT/baseURL? Phân biệt với lỗi CORS.
Table không có dữ liệu	Có đọc response.data.data? API thật trả {success,message,data}?
401 dù có token	Bearer header, token còn hạn, đúng login response và isAuth?
Student xóa được	Route DELETE có isAuth/isAdmin thật hay chỉ ẩn nút ở React?

PROMPT 12 · Chẩn đoán lỗi chính xác

C · CONTEXT Tôi đang ở Bước [số] trên project Backend cũ + frontend Vite React.

T · TASK Phân tích lỗi: [method, URL, status, response, Console/Network, đoạn file thực tế].

C · CONSTRAINT Không đổi stack, không tắt CORS/JWT/Validation, không tạo Backend mới và không đoán file thiếu.

O · OUTPUT Nguyên nhân có bằng chứng, diff file cần sửa nhỏ nhất, test tái hiện và test hồi quy.

MẪU PROMPT-LOG

Bước | Prompt đã dùng/đã sửa | AI đề xuất gì | Tôi chấp nhận/từ chối gì | Test thực tế | Kiến thức rút ra

20 / NỘP BÀI

Hồ sơ bắt buộc và thang chấm 10 điểm

Giáo trình ưu tiên sản phẩm chạy được, có bằng chứng từ Request tới SQL Server, thay vì chỉ nộp code AI tạo ra.

Sinh viên nộp

- Một repository backend-quan-ly-sinh-vien đã nâng cấp, gồm thư mục frontend/ React Vite; không nộp node_modules, dist hoặc .env.
- Script sql/05-add-avatar-url.sql; ảnh SSMS cho thấy User.avatarUrl; ảnh thư mục upload chỉ chứa file test.
- docs/openapi.yaml cập nhật; ảnh Swagger có upload multipart + ổ khóa Bearer.
- Postman Collection đã xóa token/bí mật thật; docs/test-results.md ghi U01–U12 và F01–F12 (PASS/FAIL trung thực).
- docs/prompt-log.md với ít nhất 6 prompt thật đã dùng, ít nhất 2 lần kiểm tra/sửa code AI.
- Video demo ngắn hoặc ảnh trình tự Login → GET Table → Admin DELETE → Student upload Avatar → Refresh Profile.

Gợi ý rubric 10 điểm

Tiêu chí	Điểm	Minh chứng tối thiểu
Upload Multer + giới hạn + kiểm nội dung	2.5	U01, U03, U07, U08, file hợp lệ và rác bị chặn.
SQL Server + JWT và đúng User	2.0	avatarUrl trong DB; A/B không bị sửa chéo; Profile 200/401.
Vite/React + Axios + CORS	2.0	Table thật, Loading/Empty/Error, Network GET và CORS.
Login + Admin DELETE + Avatar UI	2.0	401/403/200, 1 lần upload end-to-end.
Swagger, test, prompt-log, giải thích	1.5	YAML, 24 ca có Actual và review AI.

KHÔNG ĐƯỢC LÀM

Không đạt yêu cầu trọng tâm nếu upload không kiểm JWT, cho client sửa avatar User khác, xóa được bằng Student, chỉ dùng mock data, lưu ảnh nhưng không cập nhật DB, hoặc vô hiệu hóa CORS/auth để "sửa lỗi".

21 / TỰ KIỂM TRA

12 câu hỏi bảo vệ bài thực hành

Sinh viên trả lời bằng lời của mình và chỉ ra file tương ứng trong project. Không đọc lại nguyên câu trả lời AI.

01 application/json khác multipart/form-data ở điểm nào? Vì sao express.json() không đủ cho file?

02 Điều gì xảy ra khi FormData dùng photo nhưng Multer cấu hình single("avatar")?

03 req.file và req.body nằm ở đâu sau Multer? req.user xuất hiện ở bước nào?

04 Vì sao gọi isAuth trước Multer? Có cần isAdmin để User đổi Avatar chính mình không?

05 Vì sao không nhận userId trong body cho endpoint /me/avatar?

06 Ổ đĩa chứa dữ liệu nào, còn dbo.Users.avatarUrl chứa giá trị gì?

07 2MB tương ứng bao nhiêu byte? File 2.200.000 byte nên trả status nào?

08 Chỉ kiểm mimetype và .png có đủ để khẳng định file là ảnh PNG hợp lệ không?

09 CORS kiểm soát điều gì? Vì sao Postman gọi API được nhưng browser lại báo CORS?

10 Axios GET /api/v1/sinh-vien nên đọc mảng ở response.data hay response.data.data?

11 Vì sao Student bị ẩn nút Xóa vẫn phải bị chặn DELETE ở Backend?

12 Trình bày ba bằng chứng cần có để chứng minh upload Avatar thật sự thành công.

Bài tập mở rộng (không thay thế bài chính)

- Thiết kế cơ chế thu dọn Avatar cũ an toàn khi User upload lần tiếp theo; xử lý lỗi DB tránh tệp mồ côi.
- Đề xuất phương án chuyển nơi lưu file từ ổ đĩa sang object storage/cloud nhưng giữ SQL Server chỉ lưu URL; nêu lý do.
- Bổ sung progress bar upload và AbortController/Cancel cho Axios, giải thích khi nào dùng.

QUY TRÌNH AI-DRIVEN CẦN TỰ CHỊU TRÁCH NHIỆM

HIỂU YÊU CẦU → VIẾT PROMPT CTCO → ĐỌC & REVIEW CODE → CHẠY BUILD → KIỂM THỬ THẬT → GIẢI THÍCH KẾT QUẢ